

Playwright Promises Masterclass Guide

A Low-Level Presentation Blueprint for Technical Teams

Slide 1: Introduction & The Core Mechanics

- **Concept:** A Promise is not a text string like 'pending'. It is a real C++ object allocated in the V8 engine heap memory.
- **Internal Slots:** It holds `[[PromiseState]]` (0: Pending, 1: Fulfilled, 2: Rejected) and `[[PromiseResult]]`.

Slide 2: The Mid-Presentation Core Question

Is the 'await' waiting period considered Parallel Execution?

- **The Answer:** No! Your Test Code is Single-Threaded. It completely pauses and yields the CPU (Idle).
- **Where Tightly Scoped Parallelism Happens:** It occurs exclusively inside the Browser Architecture (C++ / OS level), completely decoupled from the Node.js test runner thread.

```
+-----+ | [Node.js  
JavaScript Thread] | | `await page.click()` ---> (Pauses JS Execution - IDLE)|  
+-----+ |-----+ | (WebSocket  
Network Signal) v +-----+  
| [Browser OS Processes] (Chromium / Firefox / WebKit) | | * Executes click on  
separate C++ background threads | | * Parallelism happens HERE, not in your  
test code! | +-----+
```

Slide 3: Network Transport Layer (JSON-RPC)

- Playwright communicates over persistent **WebSockets** using the **JSON-RPC** protocol.

```
{ "id": 42, "method": "Runtime.evaluate", "params": { "expression":  
"document.querySelector('#btn').click()" } }
```

Slide 4: Cross-Browser Abstraction Architecture

Browser	Engine	Low-Level Protocol
Chromium	Blink / V8	Native CDP (Chrome DevTools Protocol)
Firefox	Gecko / SpiderMonkey	Patched CDP Protocol by Microsoft
WebKit	WebKit / JSCore	WebKit Remote Automation RPC (Apple)